

PENDULUM DIGITAL · BUILD STANDARD

STANDARD OPERATING PROCEDURE

TWO BUILDS, ONE CODEBASE

STATIC PAGES AND A SINGLE-PAGE APP, FROM ONE CODEBASE

How to build a site that search engines can read and visitors can move through instantly.
Written from a production build. Includes the seven faults that actually occurred.

WHAT THIS IS

Most sites pick one of two shapes. A static site gives every page a real URL and real HTML, which search engines and link previews need, but every click reloads the whole page. A single-page app makes navigation instant, but the server sends an empty shell and everything is assembled in the visitor's browser — so a crawler, a link preview or a visitor on a slow connection may see nothing at all.

This procedure builds both at once. Every route is rendered to a real HTML file at build time, so the first hit is a finished page. The app then takes over in the browser, and every click after that is instant with no round trip. One codebase, one route list, both behaviours.

WHY IT IS WORTH THE EXTRA STEP

First load is a finished page	The server sends complete HTML with the copy, the headings and the images already in it. Crawlers index it, link previews render it, and someone on a poor connection sees content before any JavaScript has run.
Every click after that is instant	Once the app has taken over, navigation is a client-side swap. No round trip, no white flash, no re-downloading the header and footer on every page.
Every page still has its own URL	Each route is a real file on disk with its own title, description, canonical and structured data. It can be linked to, shared, bookmarked and indexed independently.
It fails loudly, not silently	The build refuses to publish if a page would go out with the wrong canonical URL. That one check is what stops the most expensive failure in this architecture.

THE STACK

Any equivalent will do. The pattern matters, not the libraries.

Framework	React 19
Build tool	Vite 7
Language	TypeScript
Routing	react-router-dom 7
Server render	react-dom/server renderToString + StaticRouter
Hosting	Any static host. Vercel, Netlify and Cloudflare Pages all work

THE PROCEDURE

Eight steps. Step one is the one everything else depends on.

1 ONE ROUTE TABLE, IMPORTED EVERYWHERE

Create a single file that exports the list of routes. The browser router, the server renderer and the page generator all import THAT FILE. None of them keeps its own copy.

This is the whole architecture in one decision. Skip it and the three lists drift, and the drift is silent — see trap 1.

2 A SERVER ENTRY THAT RENDERS ANY URL TO A STRING

Write an entry that takes a URL, renders the app inside StaticRouter, and returns the HTML plus whatever the page recorded for its head: title, description, canonical, structured data. Re-export the route list from it.

Use eager imports here, not lazy ones. The server pass has no Suspense boundary to fall back to.

3 A HEAD STORE THE SERVER CAN READ

Components cannot write to document.head on the server. Have each page call a small recorder during render, and have the server entry collect what was recorded. In the browser, the same component writes the tags in an effect.

Call the recorder during render, not in an effect. Effects never run during a server render.

4 A GENERATOR THAT WRITES ONE HTML FILE PER ROUTE

After the client and server bundles are built, import the server bundle, loop the route list, call render for each one, inject the head tags into the HTML template, and write the file to its own folder.

Read the route list out of the built bundle, not from a hand-written array in the script.

5 THE CANONICAL GUARD

For every route, compare the canonical the page rendered against the route being rendered. If they differ, collect the problem. If any problems exist at the end, print them and exit non-zero so the build fails.

This is the check that catches a route the generator knows about but the router does not. Without it that page publishes as the 404 page and the build still reports success.

6 GENERATE THE SITEMAP AND ROBOTS FROM THE SAME LIST

While looping the routes, collect the indexable ones and write sitemap.xml and robots.txt. Exclude anything marked noindex.

Generated, never hand-maintained. A hand-written sitemap goes stale the first time someone adds a page.

7 AN IMAGE MANIFEST

Have the asset pipeline write a file listing every image and the widths that actually exist. Components name an image, never a filename, and a helper builds the srcset from the manifest.

Stops a component requesting a size that was never generated — see trap 3.

8

DEPLOY AS A STATIC SITE

Point the host at the output folder. No server runtime is needed. Enable clean URLs so `/about` serves `/about/index.html`.

If the app lives in a subfolder of the repo, set the host's root directory to that folder — see trap 2.

BEFORE YOU CALL IT DONE

Seven checks. Each catches a failure that looks like success.

1. View source on a built page. The copy and headings must be in the HTML, not just in the JS bundle.
2. Disable JavaScript and load three pages. All three must render.
3. Check every route's canonical matches its own URL.
4. Click between pages with the network tab open. There should be no document request after the first.
5. Break a route on purpose and confirm the build fails rather than publishing.
6. Load the site on a phone and check no page scrolls sideways.
7. Watch the browser console on five routes for hydration warnings.

TRAPS

Every one of these happened on a real build. None announced itself.

THREE ROUTE LISTS THAT DRIFT

The template this came from kept the routes in three places: the browser router, the server renderer and the generator. A route present in two of them but missing from the third fell through to the wildcard, and its page was written as the 404 page — noindex, wrong canonical, none of the real content — while the build printed a tick beside it.

What it cost: Three live pages shipped broken and nobody noticed. One exported array, imported by all three, plus the canonical guard.

HOST BUILDS THE WRONG FOLDER

Where the app lives in a subfolder, a host with no root directory set builds the repository root instead. It finds nothing, produces an empty deployment in a few seconds, and that empty deployment takes over the live URL.

What it cost: Production served 404 on every route about thirty seconds after each push, which looked fine immediately after a manual deploy and broken shortly after. Set the root directory.

HARDCODED IMAGE WIDTHS

Writing `image-1200.webp` directly in a component 404s for every source narrower than 1200px, and the failure only appears in the browser console.

What it cost: Fixed twice, because it was reintroduced in a second component after the manifest existed. Components name a stem; the manifest supplies the widths.

CLASS NAME COLLISIONS IN GLOBAL CSS

A flat global stylesheet lets two unrelated components share a class name. One component's rule then applies to the other, and only some of it — enough to be invisible in review.

What it cost: Three occurrences in one build. A connector line drawn across every photo gallery; and a heading wrapper that inherited `position: absolute` from a canvas overlay and stacked every section title on top of the hero. Scope your class names.

BACKDROP-FILTER CREATES A CONTAINING BLOCK

An element with `backdrop-filter` becomes the containing block for any `position: fixed` descendant. A full-screen overlay nested inside it sizes to the parent instead of the viewport.

What it cost: A mobile menu painted a dark band the height of the header with its links spilling out transparent below. Render overlays as siblings of the filtered element, not children.

AUTO GRID TRACKS REFUSE TO SHRINK

A grid track sized auto is floored at its content's min-content width. One child that will not shrink sizes the whole track, and it can exceed the viewport.

What it cost: A panel measured 448px inside a 390px screen and pushed a control off the edge. Use `minmax(0, 1fr)` and `min-width:0` on flex and grid children that contain scrollable rows.

A FAILED BUILD LEAVES A MIXED OUTPUT FOLDER

If a build fails partway, the output can hold pre-rendered HTML from one build beside a JavaScript bundle from another. The two disagree and the browser reports hydration errors on every page.

What it cost: Chased as a code fault before the cause was found. Always confirm the build exited zero before testing the output.

BRIEF FOR AN AI AGENT

Paste this whole block. The final paragraph is the part most agents skip, so leave it in.

Build a React site that is server-rendered at build time and a single-page app after that. React 19, Vite, TypeScript, react-router-dom.

Requirements:

1. A single routes file exporting the route array and the list of paths to pre-render. The browser router, the SSR entry and the pre-render script must all import that file. Do not let any of them keep its own copy of the routes.
2. An SSR entry exporting `render(url)` that renders the app in `StaticRouter` and returns `{ html, head }`, where `head` carries `title`, `description`, `canonical`, `noindex`, `social image` and `JSON-LD`. Re-export the pre-render path list from it. Use eager imports, not lazy.
3. A head recorder that pages call DURING RENDER, not in an effect, so the server pass can collect it. The same component writes real tags to `document.head` in an effect for the client.
4. A pre-render script that imports the built SSR bundle, renders every path, injects the head into the HTML template, and writes each route to its own `index.html`.
5. A canonical guard in that script: if any rendered canonical does not match the path being rendered, print the mismatches and exit non-zero so the build fails. This catches a route missing from the router.
6. Generate `sitemap.xml` and `robots.txt` from the same route list, excluding `noindex` routes.
7. An image manifest generated by the asset pipeline listing each image and the widths that exist. Components reference an image by name and a helper builds the `srcset`. Never hardcode a width in a component.
8. Scope CSS class names per component. Do not reuse a generic name like `.rail` or `.card` in a global stylesheet.

Verify before you tell me it is done: view-source on a built page contains the real copy; every canonical matches its own route; removing a route from the router makes the build fail; no page scrolls sideways on a 390px viewport.

When not to use this. This procedure produces a site with no server runtime: the output is plain files, so hosting is cheap, fast everywhere and has very little that can break. The trade is that content changes require a rebuild, which takes seconds and happens automatically on push. If a site needs content that changes by the minute, or pages personalised per visitor, this is the wrong shape and you want server rendering on request instead.